

claude-windows / 45ea3a91-654d-4972-9cd3-65f35db54caf

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\45ea3a91-654d-4972-9cd3-65f35db54caf\subagents\workflows\wf_2a64bc88-2a8\agent-aa35923bfb4cdb124.jsonl`  
- SHA-256: `5f96860e97e5dc1d60fa28774d75f4faa6fe8fedff3aac32189a6d206f8bb895`  
- Source modified: `2026-06-10T06:14:36+00:00`  
- Imported at: `2026-07-05T16:48:27+00:00`  
- project: `wf_2a64bc88-2a8`  
- session_id: `45ea3a91-654d-4972-9cd3-65f35db54caf`
```

Transcript

1. user (2026-06-10T06:12:09.342Z)

CONTEXT ? two sibling repos on a Windows machine:

- INDEXER: C:/Users/avidu/Projects/badminton-highlight-indexer ? local AI badminton(->racket-sports) highlight indexer + rally detector. FastAPI + SQLite + ffmpeg + GPU CV via WSL; Gemini = paid cloud AI. docs/ tree is rich. Current branch docs/next-steps-multisport (master = main branch).

- COLLECTOR: C:/Users/avidu/Projects/sports-data-collector ? golden-label acquisition/curation/ingestion CLI (system-of-record candidate for training corpus).

Project state (June 2026): scaffolding complete; owned-model roadmap agreed (docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md) but NO platform/owned-model implementation started; next committed platform slice = async job model. Licensing guardrail: MIT/Apache-2.0/BSD only for code+data+weights; paid LLMs are inference-only, never training-data sources. ENVIRONMENT RULES: this dev environment has NO GPU/torch/cv2 ? do NOT attempt to run the video pipeline or import torch/cv2 modules. Test suites are offline/fully-mocked and SAFE to run. You are READ-ONLY: never edit/create/delete repo files; running tests and git read commands is allowed.

QUALITY BAR: cite file paths (and line numbers where possible) for every finding. Report what IS in the code/docs, not what you assume. Be skeptical and concrete. Your final structured output is consumed programmatically by an orchestrator ? return raw data, not prose for a human.

ROLE: adversarial verifier. An auditor ("docs-technical") claimed the finding below. Try to REFUTE it against the actual files. Default to isReal=false if the evidence does not hold up, the behavior is documented/deliberate (e.g. listed in docs/CODE_MAP.md gotchas or DEFERRED_HARDENING_PLAN.md as a known deferral ? unless the auditor adds genuinely NEW consequence), or the severity is materially overstated.

FINDING: {"title":"CODE_MAP documents `indexing.wasb.\*` config knobs that are silently dropped by the typed config in live runs","severity":"high","area":"docs/CODE_MAP.md (spot-check) + config plumbing","file":"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\CODE_MAP.md:110,266","evidence":"CODE_MAP §2.1: wasb_hybrid 'Config `indexing.wasb.velocity\_threshold(0.02)`'; §7 ramp path: 'cfg `config.json indexing.{wasb,tracknet}.velocity\_threshold`. Code reality: backend/config/models.py::IndexingConfig declares `tracknet: TrackNetConfig` but NO `wasb` field, and IndexingConfig has default pydantic extra='ignore' (only AppConfig sets extra='allow'). Verified empirically: AppConfig(indexing={'wasb': {'velocity_threshold': 0.99}}).get('indexing') -> 'wasb' key absent, while tracknet survives. wasb_hybrid.py:89 `w = idx\_cfg.get(\"wasb\", {})` therefore always sees {} when fed the live typed config -> velocity_threshold/weights_path/conda_env overrides in config.json silently do nothing (defaults from wasb_runner.py:31-51 always used).\",\"why_it_matters\":\"The cold-start map tells engineers to tune a knob that is a no-op in the live pipeline ? both a doc error and a latent code bug (WasbConfig.from_indexing_cfg only works when handed a raw dict, as in tests/eval drivers).\"} Steps: open the cited file(s) and any counter-evidence files; verify quotes/line claims; judge severity honestly. Cheap greps allowed; do NOT run the pipeline. Return isReal, confidence, and a 1-3 sentence verdict (with corrected detail if partially right).

2. user (2026-06-10T06:12:17.838Z)

```

95     - ? `::AppConfig.get` ? legacy dict-compat shim: returns nested **sections as plain
dicts** (`model_dump(mode='json')`, NOT the model) so chained
`config.get("indexing",{ }).get(...)` keeps working; falls back to searching a leaf key in any
section (sections discovered dynamically from `model_fields`). ? "bit every validator on real
runs" per docstring ? returning the model breaks `.get`. `::AppConfig.__getitem__` raises
`KeyError` for unknown top-level keys else delegates to `.get`.
96     - `::IndexingConfig` ? segmenter/detector knobs (`motion_threshold=0.08`,
`min_segment_duration=3.0`, `dead_time_merge_gap=2.0`, `chunk_duration=90.0`,
`chunk_overlap=10.0`, `yolo_velocity_threshold=0.15`, `yolo_frame_skip=3`,
`ai_handoff_padding=2.0`, `ai_max_workers=5`, `log/store_yolo_candidates=True`, nested
`tracknet`). ? `default_segmenter='yolo_hybrid'` HERE but config.json overrides to `gemini`. ?
`::IndexingConfig.segmenter_must_be_registered` `@field_validator` is a **no-op pass-through** ?
a typo'd segmenter validates fine, only fails later at `segmenter_registry.get(...)` .
97     - `::ValidationConfig` (`min_resolution_width=1280`, `min_resolution_height=720`, `min_fps=29.9`, `min_blur_threshold=20.0`, `max_scene_cuts_per_minute=0.5`, nested `relevance`);
`::ValidationRelevanceConfig` (court HSV gates, `court_pixels_min_ratio=0.05`).
`::TrackNetConfig` (WSL invocation contract: `wsl_distro='Ubuntu'`, `repo_dir`,
`conda_env='TrackNetV4'`, `weights_path='', `queue_length=5`, `velocity_threshold=0.02`).
`::OutputConfig` (`default_highlights_name`, `db_name='sports_indexer.db'`,
`output_dir='output'` ? CLI `--output-dir` overrides it on the typed model in `main()`).
`::StitchingConfig`
(`begin_padding=0.5`, `end_padding=0.5`, `use_cache=False`, `min_shot_count=1`). `::Sport` Literal
of 5 sports. `::Config` = `AppConfig` alias.
98     - `@backend/config/loader.py` ? load/save with defaults-merge.
99     - `::DEFAULT_CONFIG_PATH` = Path("config.json")` (? CWD-relative, NOT repo-root).
`::get_builtin_defaults` = `AppConfig().model_dump(...)` .
100    - `::load_config` ? precedence built-in-defaults -> file overrides. ? **shallow top-
level merge** (`{**defaults, **file_data}`): a section in config.json REPLACES the whole
defaults dict for that section (Pydantic then re-applies field defaults for missing leaves).
Missing/unreadable/parse-error file -> fully-defaulted AppConfig + `[CONFIG WARNING]` (non-
fatal).
101    - `::save_default_config` ? writes fresh `config.json`, **overwrites** if present
(used by setup.py/--setup). `::get_default_config` ? ? transitional plain-dict alias.
102    - `@config.json` ? on-disk config; mirrors `AppConfig`. ?
**`indexing.default_segmenter='gemini'`** diverges from model default `yolo_hybrid` ? file
wins at runtime.
103    - `@setup.py` ? `::init_default_config` ? delegates to
`backend.config.save_default_config` (single-source defaults; skips if config.json exists).
`::run_pip_install` GPU-aware (`nvidia-smi` -> cu124 else cpu). `::run_diagnostics` (Python>3.8,
ffmpeg/ffprobe on PATH, optional torch+CUDA).
104
105    ### 2.1 Segmenters (under pipeline/segmenters) ?
106    - `@backend/pipeline/segmenters/base.py`
107    - `::BasePlaySegmenter` ABC ? `__init__(db, config)`; abstract
`name`, `description`, `process_video`. $ `process_video(video_id, video_path, player_pool=None,
max_frames=None)` -> List[dict{id, start_time, end_time, duration, metadata}]`.
108    - `::SegmenterRegistry`, `::segmenter_registry` (singleton). ? **register a new
segmenter**: subclass `BasePlaySegmenter`, call `segmenter_registry.register(MyCls)` at module
bottom, AND import the module in `@backend/pipeline/segmenters/__init__.py` (registration is an
IMPORT side-effect). ? `register()` builds `cls(None, {})` just to read `name` ->
name/description/`__init__` MUST NOT touch `self.db`/`self.config`. ? `list_available()` re-
instantiates + swallows exceptions into "No description available." (a throwing `description` is
masked).
109    - `@backend/pipeline/segmenters/__init__.py` ? imports all 5
(`motion, gemini, yolo_hybrid, tracknet_hybrid, wasb_hybrid`) = the de-facto registration manifest;
re-exports them + `segmenter_registry` via `__all__`.
110    - `@backend/pipeline/segmenters/wasb_hybrid.py::WasbHybridSegmenter` (name
`wasb_hybrid`) ? WSL WASB substrate (MIT; ships pretrained badminton weights, the LIVE
default). Imports `WasbRunner`/`WasbConfig` from `pipeline/detectors/wasb_runner`. Config
`indexing.wasb.velocity_threshold(0.02)` + shared
`dead_time_merge_gap/min_action_window_duration/ai_handoff_padding/ai_max_workers`. ? whole-
video single pass; reuses YOLO-named keys `log_yolo_candidates/store_yolo_candidates`; needs
`{PROVIDER}_API_KEY` (missing -> []); `detection_params` has NO `queue_length` (unlike
tracknet). ? divergent shot_count: `int(shot_count)` inside try with no `is None` short-circuit
(copy-paste drift).

```

```

111 - `@backend/pipeline/segmenters/tracknet_hybrid.py::TrackNetHybridSegmenter` (name
`tracknet_hybrid`) ? copy-paste twin of wasb_hybrid; imports `TrackNetRunner`/`TrackNetConfig`
from `pipeline/detectors/tracknet_runner`; `indexing.tracknet.velocity_threshold(0.02)`;
`detection_params` adds `queue_length`. ? any P3/P4 fix must be applied to BOTH (and arguably
yolo_hybrid).
112 - `@backend/pipeline/segmenters/yolo_hybrid.py::HybridYoloSegmenter` (name
`yolo_hybrid`, no YOLO remains ? torchvision detector + supervision ByteTrack; ADR-005).
`::PERSON_CLASS_ID=1` ? (ultralitics used 0; wrong value -> zero detections).
`::lazy_load_model` (lazy `torchvision` import so module/tests load without it),
`::extract_action_windows_from_chunk`, `::make_tracker` (per-chunk ByteTrack ? track IDs
chunk-local), `::DETECTOR_FACTORIES`/`::DEFAULT_DETECTOR='fasterrcnn_resnet50_fpn'`. ?
velocity at `effective_fps = fps/frame_skip` -> `yolo_velocity_threshold` coupled to
`frame_skip`; pipelined-vs-sequential P2 (prefetch ffmpeg overlaps sequential GPU);
`yolo_tracker`/`bytetrack.yaml` is dead/back-compat. ? `gemini_*` config keys are deprecated
aliases still honored.
113 - `@backend/pipeline/segmenters/gemini.py::GeminiPlaySegmenter` (name `gemini`) ?
pure-AI, no P2. ? chunks only if `duration>chunk_duration`; `max_workers=5` hardcoded;
`max_segment_duration=90.0` hardcoded; `debug_duration` silently truncates; `parse_time` accepts
colon timestamps (warns); does NOT persist candidate segments (only hybrids feed the fine-tuning
table).
114 - `@backend/pipeline/segmenters/motion.py::MotionPlaySegmenter` (name `motion`) ?
pixel absdiff baseline. ? **metadata + serve/smash/foul/winner events are RANDOM**
(random.sample/randint/uniform/choice); only segmenter using `db.add_event` and passing
server/receiver; uses `print()`; honors `max_frames`.
115
116 ### 2.2 Detectors & windowing (WSL-quarantined; ABC abstraction complete)
117 - `@backend/pipeline/detectors/base.py` ? DetectorRunner ABC (healthcheck() ->
Tuple[bool,str] never-raises; run_predict(...) -> Optional[str] CSV path). Contract deliberately
matches the pre-existing tuple returns and call sites in the hybrids. Rich module docstring
includes "how to add a new runner", Linux-native path notes (the de-risk goal), and guidance on
threading health into reports. Re-exported from `detectors/__init__.py`. This finishes the WSL
de-risk seam (low-regret 6 / review item 1).
118 - `@backend/pipeline/detectors/tracknet_runner.py`
119 - `::TrackNetConfig` (+`.from_indexing_cfg`; ? key drift `wsl_distro` JSON -> `distro`
field), `::TrackNetRunner` ? runs TrackNet `predict.py` in WSL.
120 - `::to_wsl_mnt_path` (`C:\x`->`/mnt/c/x`; ? POSIX/`~` passthrough, UNC unhandled),
`::TrajectoryPoint`(frame,visible,x,y).
121 - **`::parse_trajectory_csv` @staticmethod** + **`::trajectory_to_action_windows`
@staticmethod** = MODEL-AGNOSTIC brain, re-exported verbatim by WasbRunner. ? `weights_path`
defaults `` -> `run_predict` returns None; predict.py only `.mp4/.avi` + hard-names
`<stem>_predict.csv`; `visible==False` resets prev (occlusion gap breaks track); `fps<=0->30`,
`frame_width<=0->1280` silent; `track_id_count` hardcoded 1; `timeout_sec=0` -> NO timeout
(hangs forever). `::stage_video` "OK" sentinel.
122 - `@backend/pipeline/detectors/wasb_runner.py::WasbRunner` (inherits DetectorRunner)
(+`::WasbConfig`, from `indexing.wasb`; ? `weights_path` HAS a real default
`wasb_badminton_best.pth.tar` ? no required guard) ? stages `wasb_infer.py` (`::INFER_WIN`) +
video into WSL, runs in `wasb` env. `::parse_trajectory_csv`/`::trajectory_to_action_windows` =
`staticmethod(TrackNetRunner.*)` (explicit rebind = the plugin-equivalence point). Output hard-
named `<stem>_wasb.csv`. ? `wasb_infer.py` re-staged each run (editing windows copy inert until
staged); `wsl_bash` duplicated (drift risk). Both runners now satisfy the DetectorRunner ABC
for future swappability.
123 - `@backend/pipeline/detectors/wasb_infer.py` (WSL) ? `::run` core. ? imports WASB-SBDT
repo modules + torch ? NOT importable on Windows. Reboot-resumable detector cache:
`::DET_FILE='det_raw.jsonl'`, `::MANIFEST_FILE='manifest.json'`, `::CACHE_VERSION=1`,
`::load_and_clean_cache` (reads longest CONTIGUOUS prefix where `w==line index`, rewrites to
heal crash-truncated tail), `::manifest_compatible` (? `frames_dir` abspath'd but `weights`
compared raw), `::atomic_write_text/_atomic_write_trajectory`, `::dets_to_tracker` (xy MUST be
np.array), `::build_cfg` (`runner.gpus=[0]` hardcoded), `::extract_frames` (cv2-PNG SLOW),
`::default_cache_dir` (`<stem>_wasbcache` next to out). ? GPU pass resumable; CPU tracker
STATEFUL -> always fully re-run; `--fresh` ignores cache.
124 - `@backend/pipeline/detectors/rally_gate.py` ? Gate A net-crossing post-filter (pure,
no I/O/model). `::estimate_play_axis` (larger-spread axis; ? ties -> `x`),
`::count_net_crossings` (sign changes vs median net; deadband jitter zone),
`::gate_windows`/`::apply_gate` (`min_crossings=2` default; kills single-toss service feeds).
`::GatedWindow`(start,end,crossings,visible_points,kept). ? window contract here is

```

```

(start,end)` tuples NOT the rich window dict ? caller must adapt. Consumed by calibrate_local /
distill_local / export_wasb_segments.
125
126     ### 2.3 Eval & calibration (all pure core; I/O in CLI drivers)
127     - `@backend/eval/metrics.py` ? scoring primitive (THE spine). `::interval_iou`,
`::match_intervals` (greedy, deterministic), `::score`->`::Score`(`.as_dict()` = wire shape),
`::pr_curve`, `::Match`, `::MatchResult`. ? greedy not Hungarian; empty matches -> 0.0 (not NaN)
for mean_iou/boundary errors.
128     - `@backend/eval/annotations.py` ? generic GT loader. `::load_annotations` (`start,end`
CSV; RAISES on bad rows/`start>=end`), `::parse_timestamp` (decimal or MM:SS/HH:MM:SS),
`::write_scaffold`.
129     - `@backend/eval/calibration.py` ? overfit guard. `::cross_validate` (within-video
k-fold; ? defaults `metric='recall'` deliberately), `::leave_one_video_out` (honest cross-video;
needs ?? labelled videos), `::improvement_report` (OPTIMISTIC ? selected on full GT),
`::select_best`, `::kfold_indices`, `::pct_improvement` (? from-zero -> `inf`), `::evaluate`. $
`PredictFn = Callable[[Config], List[Interval]]` = the plug-in seam. ? `generalization_gap
>=0.1` = overfit alarm.
130     - `@backend/eval/calibrate_wasb.py` ? WASB tuning CLI. `::BASELINE=(0.02,2.0,2.0)`,
`::default_grid` (45 cfigs), `::make_predict_fn` (parses trajectory once, closes over
`trajectory_to_action_windows`), `::load_golden_gts` ($`start_time,end_time` cols; ? SILENTLY
skips bad rows ? opposite of annotations.load_annotations). ? `{load_golden_gts,
make_predict_fn, BASELINE}` imported by 4 downstream drivers (calibrate_local,
export_wasb_segments, gemini_refine, audio/e1_probe).
131     - `@backend/eval/calibrate_local.py` ? distill Gemini->local thresholds (windowing +
net-crossing gate, no cloud at runtime). `::local_grid` (180 cfigs), `::make_local_predict_fn`
(adds `rally_gate.apply_gate` when `min_crossings>0`), `::build_videos` (? `SystemExit` if a
labels file yields no rallies), `::run`/`::main`. $
`LocalConfig=(velocity,merge,min_dur,min_crossings)`, `BASELINE_LOCAL=(0.02,2.0,2.0,0)`;
manifest JSON shared with distill_local.
132     - `@backend/eval/export_wasb_segments.py` ? tuned cfg -> golden/slicer CSV + comparison
+ optional clip cut. `::TUNED=(0.05,1.0,1.0)` (? from ONE video),
`::SEG_FIELDS`, `::COMP_FIELDS`, `::build_comparison`, `::seconds_to_mmss`, `::maybe_slice`
(lazy-imports rally_slicer). `--slice` requires `--video`. ? `write_segments_csv` output loads
straight into `rally_slicer.load_rally_labels` AND re-imported by gemini_refine ? schema is
load-bearing.
133     - `@backend/eval/batch.py` ? corpus aggregation (pure; orchestration is in
run_phase3_eval). `::aggregate` (micro pool TP/FP/FN, TP-weighted mean_iou, macro F1),
`::default_stages_for` (`auto`; `::FINAL_ONLY_SEGMENTERS={motion,gemini}`),
`::resolve_video_path` (normalizes collector `./data..` Windows paths), `::format_aggregate`.
134     - `@backend/eval/harness.py` ? DB-pred scorer (no re-run).
`::STAGES=('candidates','final')`, `::get_predictions`
(candidates->`query_candidate_segments(detector=)`; final->`query_play_segments({})`; ? unknown
stage raises ValueError; reused by regression.py),
`::evaluate`->`::EvalReport`(`::StageReport`)->`::report_to_dict` (input contract for
batch.aggregate), `::format_report_text`.

```

3. user (2026-06-10T06:12:18.833Z)

```

250     - **`add_video` INSERT OR REPLACE + FK CASCADE** -> re-ingesting a `video_id` cascade-
deletes its segments. Use `clear_video_data` deliberately. `_get_connection` MUST re-issue
`PRAGMA foreign_keys=ON` per connection.
251     - **Twin segmenters:** `wasb_hybrid.py` / `tracknet_hybrid.py` are copy-paste; fixes to
P3/P4 must land in both. ? subtle drift: wasb's shot_count `int(None)`-via-TypeError fallback vs
tracknet's `is None` check.
252     - **Two "annotations" modules:** `backend.eval.annotations` (GT CSV loader) vs
`backend.annotations` (provider registry). regression.py imports the latter.
253     - **calibrate_wasb.{load_golden_gts, make_predict_fn, BASELINE}` is a de-facto shared
API** for calibrate_local, export_wasb_segments, gemini_refine, e1_probe ? changing its golden-
CSV schema or `BASELINE` ripples widely. `gemini_refine.merge_overlaps` reused by
`distill_local`; `harness.get_predictions` reused by `regression.regress`.
254     - **Hardcoded tuned constants:** `calibrate_wasb.BASELINE`, `export.TUNED`,
slicer/compiler padding ? duplicated by copy, not import; re-tuning won't propagate.
255     - **timeout_sec=0` = no timeout:** a hung WSL/GPU call blocks forever.
256     - **Stitch concat assumes identical encode:** per-clip re-encode then `-c copy` concat;
if probe fails (0.0) no boundary validation.

```

```

257     - **Audio NOT in the live pipeline:** `pipeline/audio/hit_detector` +
`eval/audio/e1_probe` are a diagnostic probe (ADR-007); recall 100% but discrimination ~2.2x ->
not adopted for fusion.
258
259     ---
260
261     ## 7. RAMP PATHS (to do X -> open Y::symbol)
262     | Task | Open |
263     |---|---|
264     | Change a config default / add a config field | `@backend/config/models.py` (single
source of truth) -> regenerate via `setup.py`/`save_default_config`; ? config.json may override
(default_segmenter) |
265     | Read config in new code | `from backend.config import load_config, AppConfig`
(`@backend/config/__init__.py`); attribute access, NOT raw `json.load` |
266     | Tune candidate windowing (velocity/merge/min-dur) |
`@backend/pipeline/detectors/tracknet_runner.py::trajectory_to_action_windows`; cfg `config.json
indexing.{wasb,tracknet}.velocity_threshold`, `dead_time_merge_gap`,
`min_action_window_duration` |
267     | Add a segmenter | subclass `@backend/pipeline/segmenters/base.py::BasePlaySegmenter`;
`segmenter_registry.register(...)` at module bottom; import in
`@backend/pipeline/segmenters/__init__.py`; if AI-based add to
`@backend/reporting/harvest.py::_AI_SEGMENTERS` |
268     | Change stitching padding | `config.json stitching.{begin_padding,end_padding}`; impl
`@backend/pipeline/stitcher/compiler.py::VideoCompiler.compile_highlights`; mirror
`@backend/tools/rally_slicer.py::BEGIN_PADDING/END_PADDING` |
269     | Add a sport | subclass `@backend/sports/base.py::SportProfile`;
`sport_registry.register(...)` in `@backend/sports/__init__.py`; define prompt+HSV in new file |
270     | Add an AI provider | subclass `@backend/providers/base.py::AIVideoProvider`;
`provider_registry.register('name', cls)` |
271     | Add an annotation/GT tier | subclass
`@backend/annotations/base.py::AnnotationProvider`; register + import in
`@backend/annotations/__init__.py` |
272     | Add / change a report footgun rule | `@backend/reporting/spec.yaml::rules`
(declarative; ? `when.path` must match harvest's stage/metric/detail names); facts in
`@backend/reporting/harvest.py` |
273     | Tweak what the report records | `@backend/reporting/harvest.py::record_run` + stage
builders; emit point is `@backend/main.py` `finally:` ->
`@backend/reporting/__init__.py::emit_indexer_report` |
274     | Run calibration (WASB thresholds) | `@backend/eval/calibrate_wasb.py::main`
(`--trajectory --labels --fps --frame-width`); trust `cv` recall not `fit` |
275     | Distill Gemini -> local (thresholds / learned) |
`@backend/eval/calibrate_local.py::main` (thresholds) OR `@backend/eval/distill_local.py::main`
(learned classifier); both `--manifest` |
276     | Refine WASB candidates with Gemini (tuning) | `@backend/eval/gemini_refine.py::main`
(`--full-video` skips `--trajectory`; needs `GEMINI_API_KEY`) |
277     | Export tuned rally segments / clips | `@backend/eval/export_wasb_segments.py::main`
(`--slice` needs `--video`) |
278     | Export timelines (EDL/JSON/CSV) + basic stats (Value Layer slice) |
`backend/tools/export.py` (segments_to_edl/json/csv, basic_stats); called from anywhere that has
play_segments (CLI, notebook, future API) |
279     | Run a video end-to-end | `@backend/main.py::main` (`--video-path` or `--server`) or
`@run_process_and_stitch.py::main` (edit hardcoded paths) |
280     | Score DB preds vs GT (one video) | `@run_eval.py::main` |
281     | Batch score a collector manifest | `@run_phase3_eval.py::main` (`--manifest`) |
282     | Guard a regression (CI) | `@run_regression.py::main` (`--video-id`); exit 1=regress,
2=missing DB; snapshot via `--update-baseline` |
283     | Run the whole test suite | `@run_all_tests.py` (repo root); or `pytest tests/`
directly |
284     | Resume a crashed WASB pass | nothing to do ?
`@backend/pipeline/detectors/wasb_infer.py::run` auto-resumes from
`det_raw.jsonl`+`manifest.json` next to `--out`; `--fresh` forces restart |

```

4. user (2026-06-10T06:12:28.085Z)

6:The models support graceful migration from old config.json files via extra="allow".

```

33:class TrackNetConfig(BaseModel):
35:    repo_dir: str = "~/models/TrackNetV4"
37:    conda_env: str = "TrackNetV4"
46:class IndexingConfig(BaseModel):
67:    tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
108:    # Allow extra keys during transition so existing config.json files do not break.
110:        "extra": "allow",

```

5. user (2026-06-10T06:12:29.305Z)

```

1: """WASB-based hybrid play segmenter.
3: Identical three-phase shape to TrackNetHybridSegmenter, but Phase 2 uses **WASB**
4: (nttcom/WASB-SBDT, MIT) shuttle-trajectory detection via WSL2
5: (backend/detectors/wasb_runner.py) instead of TrackNetV4. WASB ships pretrained
25: from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
38: class WasbHybridSegmenter(BasePlaySegmenter):
41:     return "wasb_hybrid"
45:     return ("Two-stage pipeline: WASB shuttle-trajectory detection (WSL2, pretrained
badminton "
58:     sport_name = self.config.get("sport", "badminton")
65:     idx_cfg = self.config.get("indexing", {})
66:     provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider", "gemini"))
67:     model_name = self.config.get("ai_model", idx_cfg.get("ai_model", "gemini-2.5-flash"))
86:     wasb_cfg = WasbConfig.from_indexing_cfg(idx_cfg)
87:     runner: DetectorRunner = WasbRunner(wasb_cfg)
89:     w = idx_cfg.get("wasb", {})
91:     merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
92:     min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
93:     handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
94:     ai_max_workers = idx_cfg.get("ai_max_workers", 5)
95:     log_candidates = idx_cfg.get("log_yolo_candidates", True)
96:     store_candidates = idx_cfg.get("store_yolo_candidates", True)
99:     "detector": "wasb_hybrid",
103:     "weights_path": wasb_cfg.weights_path,
108:     logger.error("WASB WSL environment not ready: %s", msg)
110:     logger.info("WASB WSL env OK: %s", msg)
114:     out_dir = os.path.join("output", "wasb")
117:     logger.error("WASB produced no trajectory; aborting.")
129:     logger.info("Phase 2 (WASB) completed in %.1fs. Candidate windows: %d",
134:     logger.info(f"[WASB rally] {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
146:     video_id, detector="wasb_hybrid", start_time=cw["start"],
end_time=cw["end"],
211:     "detector": f"wasb-hybrid-{model_name}")
225: segmenter_registry.register(WasbHybridSegmenter)

```

6. assistant (2026-06-10T06:12:43.291Z)

Both core claims hold so far. Let me read models.py to confirm which class owns the `extra="allow"` config, and check how the live pipeline passes config to the segmenter.

7. user (2026-06-10T06:12:44.707Z)

```

1    """Pydantic models for the application configuration.
2
3    This module defines the canonical, validated, typed configuration for the
4    Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6    The models support graceful migration from old config.json files via extra="allow".
7    """
8
9    from __future__ import annotations
10
11    from typing import Any, Literal
12

```

```

13     from pydantic import BaseModel, Field, field_validator
14
15     Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18     class ValidationRelevanceConfig(BaseModel):
19         court_green_lower: list[int] = Field(default=[35, 40, 40])
20         court_green_upper: list[int] = Field(default=[85, 255, 255])
21         court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24     class ValidationConfig(BaseModel):
25         min_resolution_width: int = 1280
26         min_resolution_height: int = 720
27         min_fps: float = 29.9
28         min_blur_threshold: float = 20.0
29         max_scene_cuts_per_minute: float = 0.5
30         relevance: ValidationRelevanceConfig =
31         Field(default_factory=ValidationRelevanceConfig)
32
33     class TrackNetConfig(BaseModel):
34         wsl_distro: str = "Ubuntu"
35         repo_dir: str = "~/models/TrackNetV4"
36         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37         conda_env: str = "TrackNetV4"
38         weights_path: str = ""
39         queue_length: int = 5
40         stage_in_wsl: bool = True
41         wsl_stage_dir: str = "~/clips"
42         timeout_sec: int = 0
43         velocity_threshold: float = 0.02
44
45
46     class IndexingConfig(BaseModel):
47         default_segmenter: str = "yolo_hybrid"
48         use_gpu: bool = True
49         motion_threshold: float = 0.08
50         min_segment_duration: float = 3.0
51         dead_time_merge_gap: float = 2.0
52         chunk_duration: float = 90.0
53         chunk_overlap: float = 10.0
54         detector_model: str = "fasterrcnn_resnet50_fpn"
55         detector_score_threshold: float = 0.5
56         yolo_velocity_threshold: float = 0.15
57         yolo_frame_skip: int = 3
58         yolo_tracker: str = "bytetrack.yaml"
59         yolo_parallel_chunks: bool = False
60         yolo_chunk_workers: int = 2
61         min_action_window_duration: float = 2.0
62         log_yolo_candidates: bool = True
63         store_yolo_candidates: bool = True
64         ai_handoff_padding: float = 2.0
65         ai_max_workers: int = 5
66
67         tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
68
69         @field_validator("default_segmenter")
70         @classmethod
71         def segmenter_must_be_registered(cls, v: str) -> str:
72             # Registry check happens at runtime in main/segmenter selection.
73             # Here we just allow any string for forward compatibility.
74             return v
75
76

```

```

77     class OutputConfig(BaseModel):
78         default_highlights_name: str = "highlights.mp4"
79         db_name: str = "sports_indexer.db"
80         # Directory where the DB, highlight reels, and processed clips are written.
81         # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
82         output_dir: str = "output"
83
84
85     class StitchingConfig(BaseModel):
86         begin_padding: float = 0.5
87         end_padding: float = 0.5
88         use_cache: bool = False
89         min_shot_count: int = 1
90
91
92     class AppConfig(BaseModel):
93         """Root configuration object.
94
95         All runtime configuration should be accessed via an instance of this class
96         (or the loader) rather than raw dict lookups.
97         """
98
99         sport: Sport = "badminton"
100        ai_provider: str = "gemini"
101        ai_model: str = "gemini-2.5-flash"
102
103        validation: ValidationConfig = Field(default_factory=ValidationConfig)
104        indexing: IndexingConfig = Field(default_factory=IndexingConfig)
105        output: OutputConfig = Field(default_factory=OutputConfig)
106        stitching: StitchingConfig = Field(default_factory=StitchingConfig)
107
108        # Allow extra keys during transition so existing config.json files do not break.
109        model_config = {
110            "extra": "allow",
111            "validate_assignment": True,
112        }
113
114        # Temporary migration shim for legacy dict-style access (config.get("section",
115        {}).get(...)).
116        # Will be removed once all call sites (validators, segmenters, etc.) move to typed
117        # attribute access / the typed config. A nested *section* is returned as a plain
118        dict
119        # via model_dump, so legacy chained access keeps working ? returning the Pydantic
120        # section model instead breaks it (a model has no .get), which bit every validator
121        on
122        # real runs. Scalar leaves are returned as-is.
123        def get(self, key: str, default: Any = None) -> Any:
124            _MISSING = object()
125            val: Any = _MISSING
126            if key in type(self).model_fields or hasattr(self, key):
127                val = getattr(self, key, _MISSING)
128            if val is _MISSING:
129                # Legacy convenience: find a leaf key inside any nested section. Sections
130                are
131                # discovered dynamically from the model fields, so a future top-level
132                section
133                # (e.g. `audio`, `jobs`) is covered automatically without editing a
134                hardcoded list.
135                for fname in type(self).model_fields:
136                    section = getattr(self, fname, None)
137                    if isinstance(section, BaseModel) and hasattr(section, key):
138                        val = getattr(section, key)
139                        break
140            if val is _MISSING:
141                return default

```



```

136         if isinstance(val, BaseModel):
137             return val.model_dump(mode="json")
138         return val
139
140     def __getitem__(self, key: str) -> Any:
141         val = self.get(key)
142         if val is None and key not in type(self).model_fields:
143             # Re-raise like a real dict for unknown top-level
144             raise KeyError(key)
145         return val
146
147
148     # Convenience type alias for callers that want to annotate
149     Config = AppConfig

```

8. user (2026-06-10T06:13:09.295Z)

```

30:from backend.config import load_config as load_app_config, AppConfig # new typed config
59:global_config: Optional[AppConfig] = None # now typed (Pydantic model)
62:# New code should import load_app_config / AppConfig from backend.config directly.
63:def load_config(config_path: str = "config.json") -> Dict[str, Any]:
64:    cfg = load_app_config(config_path)
67:@app.get("/api/config")
68:
69:
70:
71:
72:
73:
74:
75:
76:
77:
78:
79:
80:
81:
82:
83:
84:
85:
86:
87:
88:
89:
90:
91:
92:
93:
94:
95:
96:
97:
98:
99:
100:
101:
102:
103:
104:
105:
106:
107:
108:
109:
110:
111:
112:
113:
114:
115:
116:
117:
118:
119:
120:
121:
122:
123:
124:
125:
126:
127:
128:
129:
130:
131:
132:
133:
134:
135:
136:
137:
138:
139:
140:
141:
142:
143:
144:
145:
146:
147:
148:
149:
150:
151:
152:
153:
154:
155:
156:
157:
158:
159:
160:
161:
162:
163:
164:
165:
166:
167:
168:
169:
170:
171:
172:
173:
174:
175:
176:
177:
178:
179:
180:
181:
182:
183:
184:
185:
186:
187:
188:
189:
190:
191:
192:
193:
194:
195:
196:
197:
198:
199:
200:
201:
202:
203:
204:
205:
206:
207:
208:
209:
210:
211:
212:
213:
214:
215:
216:
217:
218:
219:
220:
221:
222:
223:
224:
225:
226:
227:
228:
229:
230:
231:
232:
233:
234:
235:
236:
237:
238:
239:
240:
241:
242:
243:
244:
245:
246:
247:
248:
249:
250:
251:
252:
253:
254:
255:
256:
257:
258:
259:
260:
261:
262:
263:
264:
265:
266:
267:
268:
269:
270:
271:
272:
273:
274:
275:
276:
277:
278:
279:
280:
281:
282:
283:
284:
285:
286:
287:
288:
289:
290:
291:
292:
293:
294:
295:
296:
297:
298:
299:
300:
301:
302:
303:
304:
305:
306:
307:
308:
309:
310:
311:
312:
313:
314:
315:
316:
317:
318:
319:
320:
321:
322:
323:
324:
325:
326:
327:
328:
329:
330:
331:
332:
333:
334:
335:
336:
337:
338:
339:
340:
341:
342:
343:
344:
345:
346:
347:
348:
349:
350:
351:
352:
353:
354:
355:
356:
357:
358:
359:
360:
361:
362:
363:
364:
365:
366:
367:
368:
369:
370:
371:
372:
373:
374:
375:
376:
377:
378:
379:
380:
381:
382:
383:
384:
385:
386:
387:
388:
389:
390:
391:
392:
393:
394:
395:
396:
397:
398:
399:
400:
401:
402:
403:
404:
405:
406:
407:
408:
409:
410:
411:
412:
413:
414:
415:
416:
417:
418:
419:
420:
421:
422:
423:
424:
425:
426:
427:
428:
429:
430:
431:
432:
433:
434:
435:
436:
437:
438:
439:
440:
441:
442:
443:
444:
445:
446:
447:
448:
449:
450:
451:
452:
453:
454:
455:
456:
457:
458:
459:
460:
461:
462:
463:
464:
465:
466:
467:
468:
469:
470:
471:
472:
473:
474:
475:
476:
477:
478:
479:
480:
481:
482:
483:
484:
485:
486:
487:
488:
489:
490:
491:
492:
493:
494:
495:
496:
497:
498:
499:
500:
501:
502:
503:
504:
505:
506:
507:
508:
509:
510:
511:
512:
513:
514:
515:
516:
517:
518:
519:
520:
521:
522:
523:
524:
525:
526:
527:
528:
529:
530:
531:
532:
533:
534:
535:
536:
537:
538:
539:
540:
541:
542:
543:
544:
545:
546:
547:
548:
549:
550:
551:
552:
553:
554:
555:
556:
557:
558:
559:
560:
561:
562:
563:
564:
565:
566:
567:
568:
569:
570:
571:
572:
573:
574:
575:
576:
577:
578:
579:
580:
581:
582:
583:
584:
585:
586:
587:
588:
589:
590:
591:
592:
593:
594:
595:
596:
597:
598:
599:
600:
601:
602:
603:
604:
605:
606:
607:
608:
609:
610:
611:
612:
613:
614:
615:
616:
617:
618:
619:
620:
621:
622:
623:
624:
625:
626:
627:
628:
629:
630:
631:
632:
633:
634:
635:
636:
637:
638:
639:
640:
641:
642:
643:
644:
645:
646:
647:
648:
649:
650:
651:
652:
653:
654:
655:
656:
657:
658:
659:
660:
661:
662:
663:
664:
665:
666:
667:
668:
669:
670:
671:
672:
673:
674:
675:
676:
677:
678:
679:
680:
681:
682:
683:
684:
685:
686:
687:
688:
689:
690:
691:
692:
693:
694:
695:
696:
697:
698:
699:
700:
701:
702:
703:
704:
705:
706:
707:
708:
709:
710:
711:
712:
713:
714:
715:
716:
717:
718:
719:
720:
721:
722:
723:
724:
725:
726:
727:
728:
729:
730:
731:
732:
733:
734:
735:
736:
737:
738:
739:
740:
741:
742:
743:
744:
745:
746:
747:
748:
749:
750:
751:
752:
753:
754:
755:
756:
757:
758:
759:
760:
761:
762:
763:
764:
765:
766:
767:
768:
769:
770:
771:
772:
773:
774:
775:
776:
777:
778:
779:
780:
781:
782:
783:
784:
785:
786:
787:
788:
789:
790:
791:
792:
793:
794:
795:
796:
797:
798:
799:
800:
801:
802:
803:
804:
805:
806:
807:
808:
809:
810:
811:
812:
813:
814:
815:
816:
817:
818:
819:
820:
821:
822:
823:
824:
825:
826:
827:
828:
829:
830:
831:
832:
833:
834:
835:
836:
837:
838:
839:
840:
841:
842:
843:
844:
845:
846:
847:
848:
849:
850:
851:
852:
853:
854:
855:
856:
857:
858:
859:
860:
861:
862:
863:
864:
865:
866:
867:
868:
869:
870:
871:
872:
873:
874:
875:
876:
877:
878:
879:
880:
881:
882:
883:
884:
885:
886:
887:
888:
889:
890:
891:
892:
893:
894:
895:
896:
897:
898:
899:
900:
901:
902:
903:
904:
905:
906:
907:
908:
909:
910:
911:
912:
913:
914:
915:
916:
917:
918:
919:
920:
921:
922:
923:
924:
925:
926:
927:
928:
929:
930:
931:
932:
933:
934:
935:
936:
937:
938:
939:
940:
941:
942:
943:
944:
945:
946:
947:
948:
949:
950:
951:
952:
953:
954:
955:
956:
957:
958:
959:
960:
961:
962:
963:
964:
965:
966:
967:
968:
969:
970:
971:
972:
973:
974:
975:
976:
977:
978:
979:
980:
981:
982:
983:
984:
985:
986:
987:
988:
989:
990:
991:
992:
993:
994:
995:
996:
997:
998:
999:

```

9. user (2026-06-10T06:13:10.447Z)

```

23-
24-logger = logging.getLogger(__name__)
25-
26-# Windows path to the inference wrapper we stage into WSL.
27-_INFER_WIN = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
28-
29-
30-@dataclass
31-class WasbConfig:
32-    """Resolved settings for a WASB WSL run (built from config.json -> indexing.wasb)."""
33-    distro: str = "Ubuntu"
34-    repo_dir: str = "~/models/WASB-SBDT"
35-    conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"

```

```

36-     conda_env: str = "wasb"
37-     weights_path: str = "~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
38-     sport: str = "badminton"
39-     wsl_stage_dir: str = "~/clips"
40-     timeout_sec: int = 0 # 0 = no timeout
41-
42-     @classmethod
43-     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "WasbConfig":
44-         w = (idx_cfg or {}).get("wasb", {}) or {}
45-         return cls(
46-             distro=w.get("wsl_distro", "Ubuntu"),
47-             repo_dir=w.get("repo_dir", "~/models/WASB-SBDT"),
48-             conda_sh=w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
49-             conda_env=w.get("conda_env", "wasb"),
50-             weights_path=w.get("weights_path",
51-                                "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"),
52-             sport=w.get("sport", "badminton"),
53-             wsl_stage_dir=w.get("wsl_stage_dir", "~/clips"),
54-             timeout_sec=int(w.get("timeout_sec", 0)),
55-         )
56-
57-
58- class WasbRunner(DetectorRunner): # DetectorRunner ABC (finish item 6 / new item 1)
59-     def __init__(self, cfg: WasbConfig):
60-         self.cfg = cfg
61-
62-     def _wsl_bash(self, inner_cmd: str) -> subprocess.CompletedProcess:
63-         full = f"source {self.cfg.conda_sh} && {inner_cmd}"

```

10. user (2026-06-10T06:13:34.280Z)

59: `calibrate\_wasb.py` do honest leave-one-video-out (LOVO), but nothing *enforces*

11. user (2026-06-10T06:13:35.700Z)

No matches found

12. user (2026-06-10T06:13:36.645Z)

```

17: from backend.config import load_config
28:     print(f"Loading config...")
29:     cfg = load_config() # typed AppConfig ? single source of defaults
75:         print(f"[FAIL] Segmenter '{seg_name}' not found.")
106:     print("[WARNING] Invalid input detected, falling back to defaults from
config.json.")

```

13. user (2026-06-10T06:13:51.972Z)

wasb in idx: False
tracknet vt: 0.55
pydantic 2.13.4

14. user (2026-06-10T06:13:51.987Z)

```

84         fps, frame_width = self._probe_fps_width(video_path)
85
86         wasb_cfg = WasbConfig.from_indexing_cfg(idx_cfg)
87         runner: DetectorRunner = WasbRunner(wasb_cfg)
88
89         w = idx_cfg.get("wasb", {})
90         velocity_thresh = w.get("velocity_threshold", 0.02)
91         merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
92         min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)

```

```
93         handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
94         ai_max_workers = idx_cfg.get("ai_max_workers", 5)
95         log_candidates = idx_cfg.get("log_yolo_candidates", True)
96         store_candidates = idx_cfg.get("store_yolo_candidates", True)
97
98         detection_params = {
```

15. user (2026-06-10T06:14:31.682Z)

Structured output provided successfully

16. assistant (2026-06-10T06:14:36.020Z)

Verification complete.